The SQL Detective's Notebook

— cracking any query, one clue at a time —

field notes, case files 01–10 · filed by Claude, Query Division

Same framework as the full write-up, condensed into something you can actually glance at mid-problem. Read the clues, pick the tool, don't overthink it.

I. The Process — run this every time

before you type a single SELECT…

1. **What's ONE ROW of the output?** Say it out loud. "One row per employee." "One row per month." Everything follows from this.
2. **List the tables & joins** you'll actually need. No SQL yet — just names.
3. **WHERE, HAVING, or a wrapper?** Raw filter → WHERE. Group filter → HAVING. Filtering on a window function or a different grain → compute it first, filter one layer up.
4. **Grain mismatch check:** does the question want a group-level fact *alongside* row detail (rank, average, vs. last period)? That's your window-function signal.
5. **Sequence it as CTEs** — one transformation per step. Test each alone before stacking the next.

6

Assemble & stress-test: ties, NULLs, empty groups, "no previous row" edge cases.

II. Keyword → Technique Cheat Sheet

spot the phrase, grab the tool

"2nd highest", no LIMIT

→

DENSE_RANK / subquery

"duplicate", "how many times"

→

GROUP BY + HAVING

"vs. their dept/category average"

→

AVG() OVER (PARTITION BY)

"top N per group"

→

RANK / DENSE_RANK PARTITION BY

"running total", "cumulative"

→

SUM() OVER (ORDER BY...)

"% change vs. previous period"

→

LAG() / LEAD()

"consecutive days / streak"

→

gaps & islands

"did X in both A and B"

→

conditional agg + HAVING

"first / last event per entity"

→

ROW_NUMBER = 1

multi-step, reused logic

→

CTE chain

"compare a row to another row"

→

self-join

GROUP BY / HAVING · Window function · CTE chain · Self-join · LAG / LEAD · Gaps & islands

III. The Decision Compass

walk down this list, in order, for any new problem

? Collapsed rows, or same row-count decorated?

Collapsed → GROUP BY. Decorated → window function.

? Can I filter this straight in WHERE?

Same-grain aggregate → HAVING. Window fn / different grain → wrap it in a CTE first.

? Do I need a value from another row?

Previous / next / first / last → LAG, LEAD, FIRST_VALUE, LAST_VALUE.

? Do I need ranking or "top N per group"?

PARTITION BY + ROW_NUMBER (strict) / RANK (ties+gaps) / DENSE_RANK (ties, no gaps).

? Is the logic multi-step?

Chain it as CTEs — one transformation per step. You can't nest window fns inside each other anyway.

? Are two different grains involved?

> Compute each grain in its own CTE, then JOIN them on the shared key. Don't force one SELECT to do both.

IV. Case Files 01–10

the same 10 problems, filed and tagged

01 Second Highest Salary

Grain: one number.

Clues: "2nd highest", no LIMIT/TOP.

Window fn

```
SELECT DISTINCT salary
FROM (
  SELECT salary,
    DENSE_RANK() OVER (ORDER BY
salary DESC) rnk
  FROM Employees
) t
WHERE rnk = 2;
```

02 Dept. Average Comparison

Grain: one row per employee.

Clues: "greater than dept average" — detail kept.

Window fn CTE

```
WITH d AS (
  SELECT *, AVG(salary) OVER (
    PARTITION BY department_id)
avg_sal
  FROM Employees
)
SELECT * FROM d WHERE salary >
avg_sal;
```

03

Duplicate Emails

Grain: one row per email.

Clues: "duplicate", "how many times".

GROUP BY

```
SELECT email, COUNT(*) AS n
FROM Customers
GROUP BY email
HAVING COUNT(*) > 1;
```

04

Monthly % Change

Grain: one row per month.

Clues: "each month", "vs. previous month".

LAG GROUP BY

```
WITH m AS (
  SELECT DATE_TRUNC('month',
sale_date) mo,
    SUM(amount) total
  FROM Sales GROUP BY 1
)
SELECT mo, total,
  LAG(total) OVER (ORDER BY mo)
prev,
  ROUND((total - LAG(total) OVER
(ORDER BY mo))
    * 100.0 / LAG(total) OVER (ORDER
BY mo), 2) pct
FROM m;
```

05

Top 3 per Department

Grain: one row per employee, filtered.

Clues: "top 3 per group", ties share rank.

DENSE_RANK

```
WITH r AS (
  SELECT *, DENSE_RANK() OVER (
    PARTITION BY department_id
    ORDER BY salary DESC) rnk
  FROM Employees
)
SELECT * FROM r WHERE rnk <= 3;
```

06

Running Total

Grain: one row per day.

Clues: "running", "cumulative".

Window fn

```
SELECT sale_date, daily_sales,
  SUM(daily_sales) OVER (
    ORDER BY sale_date
    ROWS BETWEEN UNBOUNDED PRECEDING
    AND CURRENT ROW) running_total
FROM DailySales;
```

07

Retention: Jan & Feb

Grain: one row per customer.

Clues: "in both A and B".

GROUP BY

```
SELECT customer_id
FROM Orders
WHERE EXTRACT(MONTH FROM order_date)
IN (1,2)
GROUP BY customer_id
HAVING COUNT(DISTINCT
  EXTRACT(MONTH FROM order_date)) =
2;
```

08

Consecutive Logins

Grain: one row per streak.

Clues: "consecutive" → gaps & islands.

Gaps & islands

```
WITH n AS (
  SELECT user_id, login_date,
    ROW_NUMBER() OVER (
      PARTITION BY user_id
      ORDER BY login_date) rn
  FROM (SELECT DISTINCT user_id,
login_date
        FROM UserLogins) d
)
SELECT user_id,
  login_date - rn * INTERVAL '1 day'
AS anchor,
  COUNT(*) AS streak_len
FROM n
GROUP BY user_id, anchor
HAVING COUNT(*) >= 3;
```

09

Top Product per Month

Grain: one row per month.

Clues: "each month" + "highest revenue".

GROUP BY RANK

```
WITH rev AS (
  SELECT DATE_TRUNC('month',
order_date) mo,
    product_id, SUM(quantity*price)
revenue
  FROM Orders GROUP BY 1,2
), r AS (
  SELECT *, RANK() OVER (
    PARTITION BY mo
    ORDER BY revenue DESC) rnk
  FROM rev
)
SELECT * FROM r WHERE rnk = 1;
```

10

The Big One — Two Grains at Once

Grain: two grains mixed — employee totals AND per-sale running total.

Clues: rank + average + running total, all in one output.

CTE chain | Window fn | JOIN

```
WITH tot AS (
  SELECT employee_id, employee_name, department_id,
    SUM(sales_amount) total
  FROM EmployeeSales GROUP BY 1,2,3
), rnk AS (
  SELECT *, DENSE_RANK() OVER (
      PARTITION BY department_id
      ORDER BY total DESC) dept_rank,
    AVG(total) OVER (
      PARTITION BY department_id) dept_avg
  FROM tot
), run AS (
  SELECT employee_id, sale_date, sales_amount,
    SUM(sales_amount) OVER (
      PARTITION BY employee_id
      ORDER BY sale_date
      ROWS BETWEEN UNBOUNDED PRECEDING
      AND CURRENT ROW) running_total
  FROM EmployeeSales
)
SELECT rnk.*, run.sale_date, run.running_total
FROM rnk JOIN run USING (employee_id)
ORDER BY department_id, dept_rank, sale_date;
```

BEFORE YOU TYPE — THE 4-LINE SELF CHECK

1. Grain of one output row: _____
2. Keywords I spotted: _____
3. Filter type — WHERE / HAVING / needs a wrapper: _____
4. Technique(s): GROUP BY / window fn / self-join / gaps & islands / CTE chain

— case closed. now go write the query. —